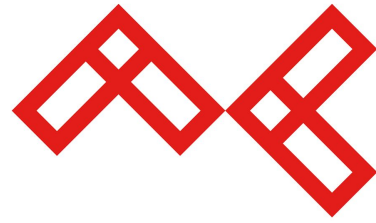
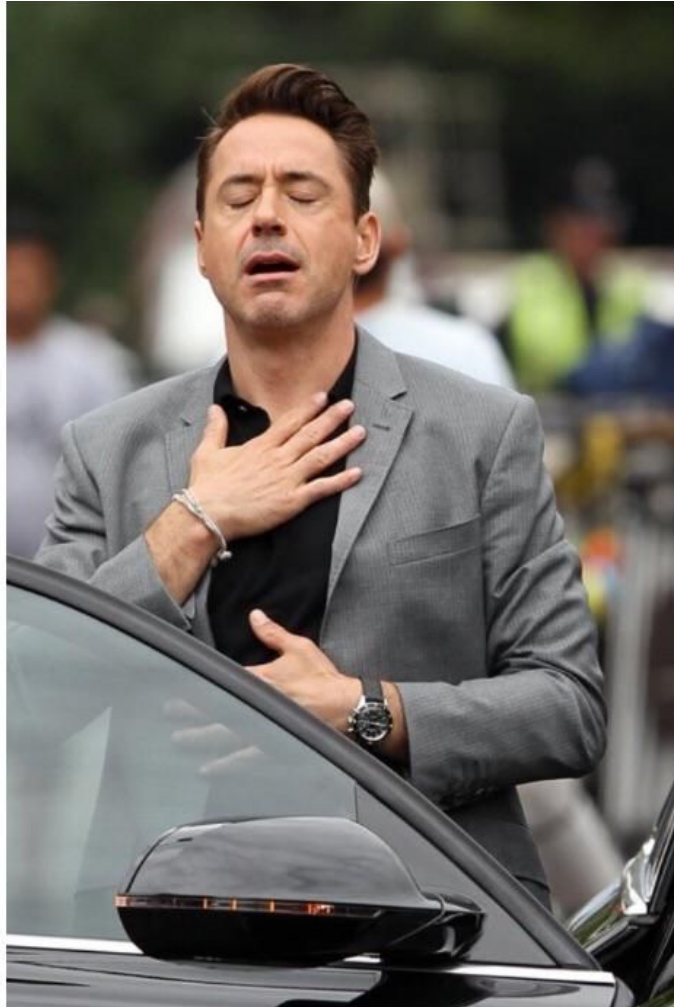


THE LA
THE FINI

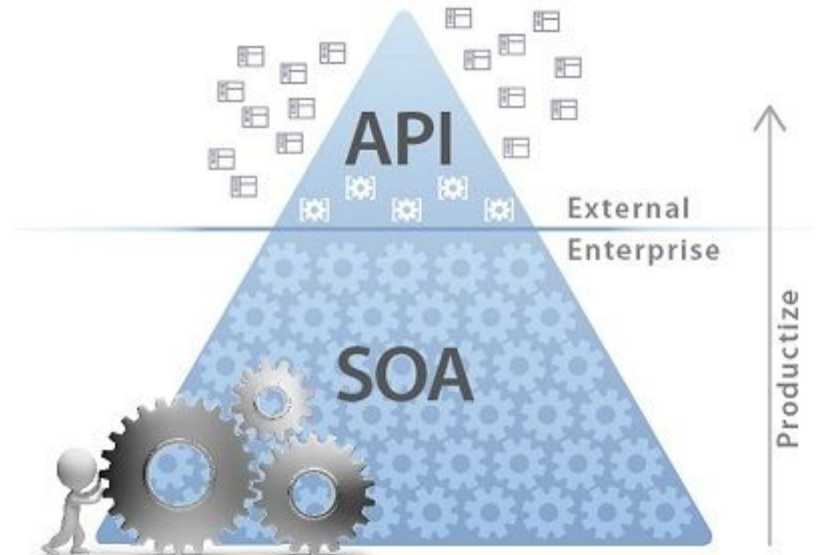
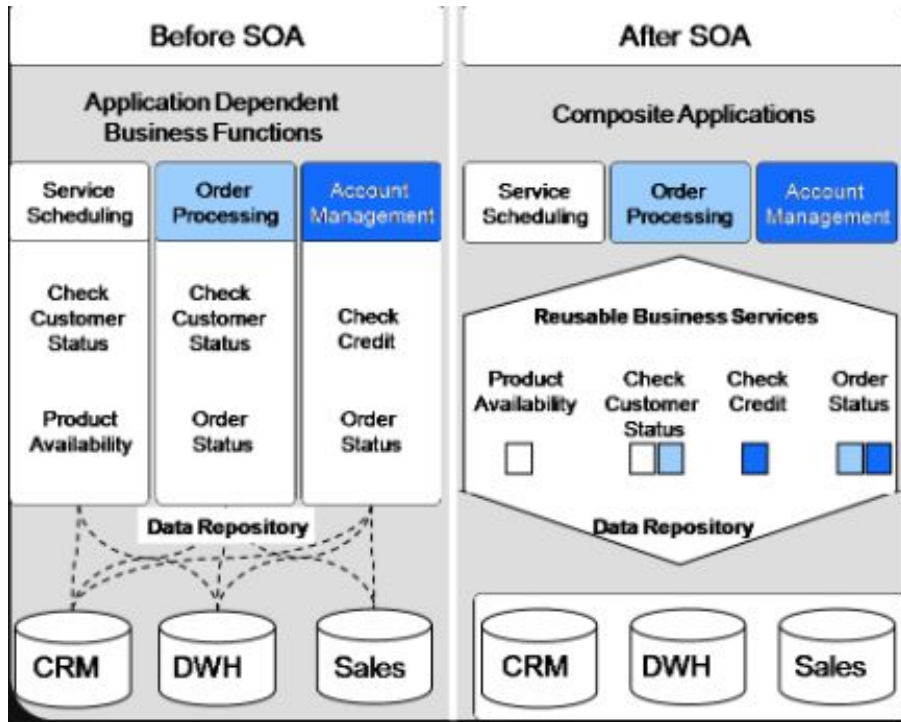
BY. JOÃO RIBEIRO



R...

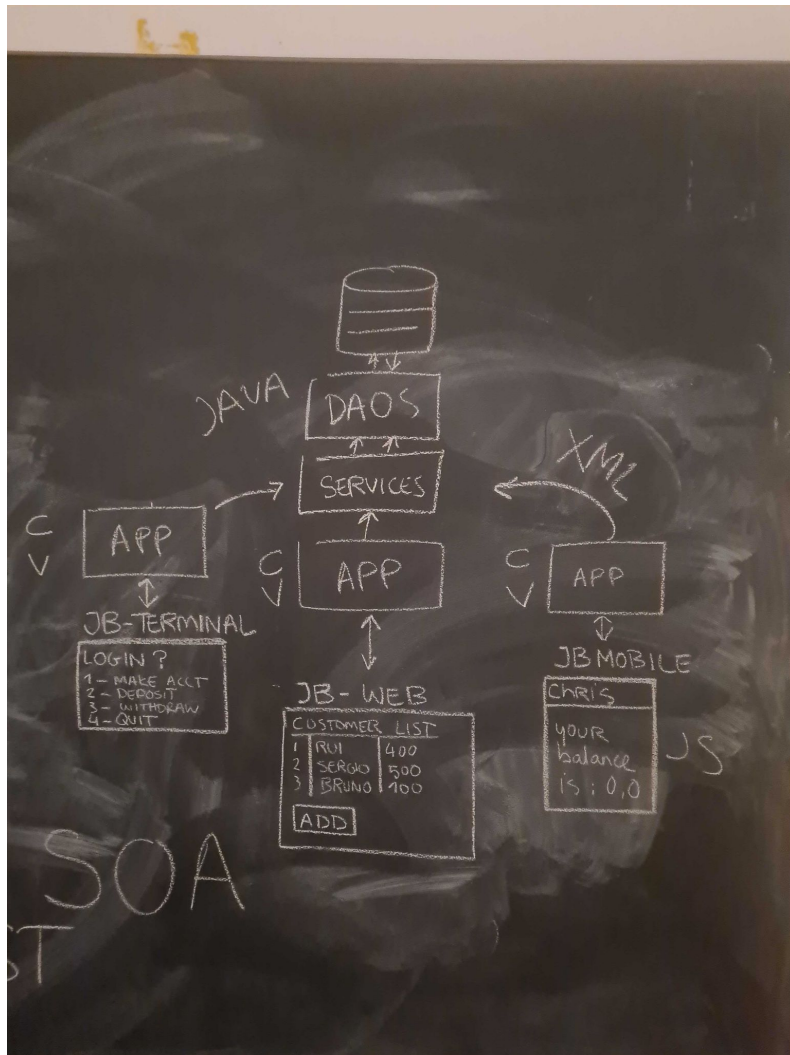
NK...

SERVICE ORIENTED ARCHITECTURE (SOA)



THE FLOW...

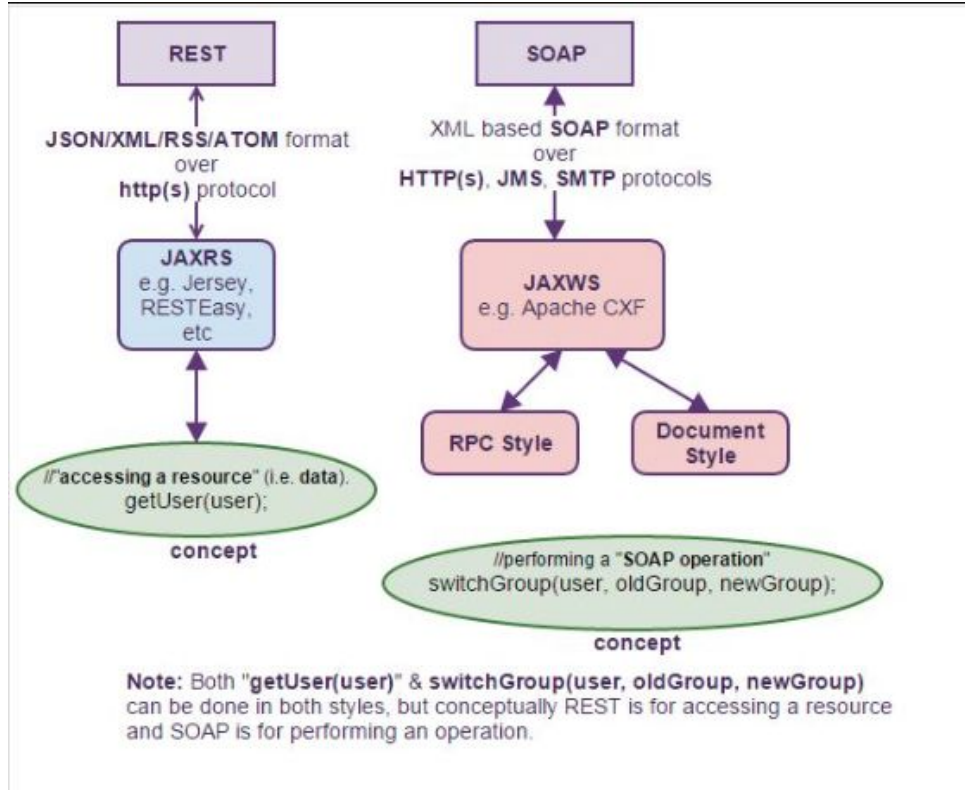
- SOAP - exposes business logic operations;
- RESTful - exposes CRUD operations on data.





WEB SERVICE

- Not a protocol
- Uses HTTP
- Stateless
- Cacheable
- Exposes CRUD
- Legacy XML
- Modern JSON



- Powerful
- Versatile
- Automated Tools
- WSDL Contract
- XML Payloads
- Too Verbose..

http://localhost:8080/Json/SyncReply/Contacts

```
{
  - Contacts: [
    - {
      FirstName: "Demis",
      LastName: "Bellot",
      Email: "demis.bellot@gmail.com"
    },
    - {
      FirstName: "Steve",
      LastName: "Jobs",
      Email: "steve@apple.com"
    },
    - {
      FirstName: "Steve",
      LastName: "Ballmer",
      Email: "steve@microsoft.com"
    },
    - {
      FirstName: "Eric",
      LastName: "Schmidt",
      Email: "eric@google.com"
    },
    - {
      FirstName: "Larry",
      LastName: "Ellison",
      Email: "larry@oracle.com"
    }
  ]
}
```

http://localhost:8080/Xml/SyncReply/Contacts

```
<ContactsResponse xmlns:i="http://www.w3.org/2003/05/soap-envelope">
  <Contacts>
    <Contact>
      <Email>demis.bellot@gmail.com</Email>
      <FirstName>Demis</FirstName>
      <LastName>Bellot</LastName>
    </Contact>
    <Contact>
      <Email>steve@apple.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Jobs</LastName>
    </Contact>
    <Contact>
      <Email>steve@microsoft.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Ballmer</LastName>
    </Contact>
    <Contact>
      <Email>eric@google.com</Email>
      <FirstName>Eric</FirstName>
      <LastName>Schmidt</LastName>
    </Contact>
    <Contact>
      <Email>larry@oracle.com</Email>
      <FirstName>Larry</FirstName>
      <LastName>Ellison</LastName>
    </Contact>
  </Contacts>
</ContactsResponse>
```

FETCH IP INFORMATION TO JSON API

```
public class Main {  
    public static void main(String[] args) {  
  
        ObjectMapper mapper = new ObjectMapper();  
  
        try {  
            Dados dados = mapper.readValue(new URL( spec: "https://ipinfo.io/json"), Dados.class);  
            String valor = mapper.writeValueAsString(dados);  
            System.out.println(valor);  
  
            System.out.println(dados.toString());  
        } catch (IOException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

THE LAST CHALLENGE OF JAVABANK...

{ Exercise }

**Add a JSON REST API with Customer CRUD operations
to the JavaBank application**

Test with Postman / curl

Http Method	EndPoint	Action
GET	/customer	read all customers, no account info
GET	/customer/{id}	read a specific customer
GET	/customer/{id}/account	read all accounts for customer
GET	/customer/{cid}/account/{aid}	read account for customer
POST	/customer	create a new customer
PUT	/customer/{id}	edit an existing customer
DELETE	/customer/{id}	delete an existing customer
DELETE	/customer/{cid}/account/{aid}	delete an account

STEP 1 - READ ALL CUSTOMERS

```
@RequestMapping(method = RequestMethod.GET, value = "/get")
public ResponseEntity <List<CustomerDto>> listClient () {

    List<CustomerDto> customerDtos = customerToCustomerDto.convert(customerService.list());

    //return "Hello " + str;
    return new ResponseEntity<>(customerDtos, HttpStatus.OK);
}
```

STEP 2 - READ A SPECIFIC CUSTOMER

```
@RequestMapping(method = RequestMethod.GET, value = "/get/{id}")
public ResponseEntity <CustomerDto> listoneClient (@PathVariable Integer id) {

    List<CustomerDto> customerDtos = customerToCustomerDto.convert(customerService.list());

    //return "Hello " + str;
    return new ResponseEntity<>(customerDtos.get(id), HttpStatus.OK);
}
```

STEP 3 - READ ALL ACCOUNTS FOR CUSTOMER

```
@RequestMapping(method = RequestMethod.GET, value = "/get/{id}/account")
public ResponseEntity <List<AccountDto>> listCounts (@PathVariable Integer id) {

    List <AccountDto> contas = accountToAccountDto.convert(customerService.get(id).getAccounts())

    //return "Hello " + str;
    return new ResponseEntity<>(contas, HttpStatus.OK);
}
```

STEP 4 - READ ACCOUNT FOR CUSTOMER

```
@RequestMapping(method = RequestMethod.GET, value = "/get/{id}/account/{aid}")
public ResponseEntity <AccountDto> listoneCounts (@PathVariable Integer id, @PathVariable Integer aid) {

    Account account = accountService.get(aid);

    if(account.getCustomer().getId() != id){
        return new ResponseEntity<>(HttpStatus.BAD_REQUEST);
    }

    AccountDto conta1 = accountToAccountDto.convert(account);

    return new ResponseEntity<>(conta1, HttpStatus.OK);
}
```

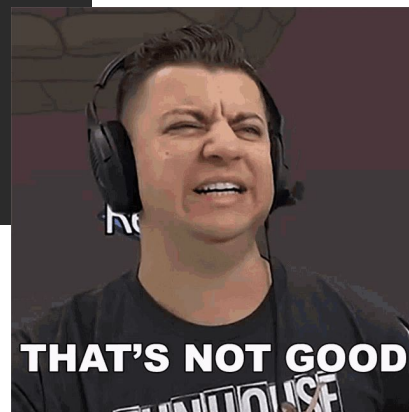
STEP 5 - CREATE A NEW CUSTOMER

```
@RequestMapping(method = RequestMethod.POST, value = "/client")
public ResponseEntity <Void> creatCustomer (@RequestBody Customer customer2) {

    Customer customer = new Customer();
    customer.setFirstName(customer2.getFirstName());
    customer.setLastName(customer2.getLastName());
    customer.setEmail(customer2.getEmail());
    customer.setPhone(customer2.getPhone());

    customerService.save(customer);

    return new ResponseEntity<>(HttpStatus.OK);
}
```



STEP 6 - EDIT AN EXISTING CUSTOMER

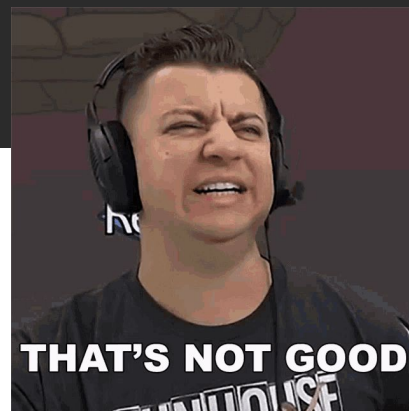
```
@RequestMapping(method = RequestMethod.PUT , value = @RequestMapping("/client/{id}")
public ResponseEntity <Void> editCustomer (@PathVariable Integer id, @RequestBody CustomerDto customer2) {

    CustomerDto customerDto1 = customerToCustomerDto.convert(customerService.get(id));

    BeanUtils.copyProperties(customerDto1, customer2, ...ignoreProperties: "id");

    customerService.save(customerDtoToCustomer.convert(customerDto1));

    return new ResponseEntity<>(HttpStatus.OK);
}
```



STEP 7 - DELETE AN EXISTING CUSTOMER

```
@RequestMapping(method = RequestMethod.DELETE , value = "/client/delete/{id}")
public ResponseEntity <Void> deleteCustomer (@PathVariable Integer id ) {

    //List<CustomerDto> customerDtos = customerToCustomerDto.convert(customerService.list());

    customerService.delete(id);

    return new ResponseEntity<>(HttpStatus.OK);
}
```



STEP 8 - DELETE AN ACCOUNT

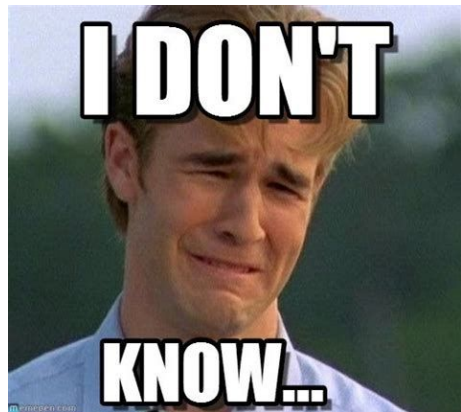
```
@RequestMapping(method = RequestMethod.DELETE, value = "/client/delete/{id}/account/{aid}")
public ResponseEntity <Void> deleteCount (@PathVariable Integer id,@PathVariable Integer aid) {

    accountService.deleteAccount(aid);

    return new ResponseEntity<>(HttpStatus.OK);
}
```

```
1 usage new *
@Transactional
@Override
public void deleteAccount(Integer id) throws AccountNotFoundException {
    Account account = Optional.ofNullable(accountDao.findById(id))
        .orElseThrow(CustomerNotFoundException::new);

    accountDao.delete(id);
}
```



NO MORE JAVABANK !!!

WE HOPE...



Thank you all !!

